DỰ ÁN GIỮA KỲ - PHÂN HỆ ĐÓNG HỌC PHÍ iBANKING

# Phân công nhiệm vụ nhóm hai người

Bản chốt 1.0.0 | Làm riêng, tự kiểm thử, sau đó ghép và hoàn thiện

Người A: ..... Người B: .....

| Người A | Người B |
|---|---|
| **Tài khoản + email + toàn bộ giao diện**<br>Account Service, Notification Service, frontend; kiểm thử và báo cáo phần A. | **Thanh toán + dữ liệu + tính nhất quán**<br>Payment Service, ví/học phí/giao dịch/OTP; transaction, concurrency, kiểm thử và báo cáo phần B. |

## Cách tổ chức để không phải chờ nhau

**A tự tạo Payment mock** để hoàn thành UI khi backend B chưa xong. **B tự tạo Notification mock và token kiểm thử** để hoàn thành backend khi A chưa xong. Cả hai dùng cùng hợp đồng API và dữ liệu mẫu đã chốt.

Ba service chạy riêng. Toàn bộ số dư, công nợ, giao dịch và OTP thuộc Payment để cập nhật tài chính trong cùng một transaction. Mỗi người có source, test, README và phần báo cáo riêng.

## Đối chiếu đủ 8 tiêu chí rubric

| Tiêu chí | Điểm | Người phụ trách chính |
|---|---|---|
| Phân tích nghiệp vụ và UML | 1,0 | A: Use Case. B: ERD. |
| Kiến trúc Microservices | 1,5 | A: sơ đồ. Cả hai: service và giao tiếp thật. |
| Thiết kế REST API | 1,0 | Mỗi người API thuộc service mình. |
| Database và persistence | 1,0 | A: Account DB. B: Payment DB. |
| Chức năng và payment workflow | 2,0 | A: login/UI/mail. B: nghiệp vụ tài chính/OTP. |
| Transaction và concurrency | 1,5 | B hiện thực; A chạy kiểm chứng. |
| Giao diện và tích hợp | 0,5 | A chủ trì; B bảo đảm Payment API. |
| Tài liệu, triển khai và demo | 1,5 | Cả hai phần mình; A ghép, B thử chạy lại. |

**Nguyên tắc nghiệm thu:** hoàn thành = code chạy được + kiểm thử phù hợp + tài liệu tương ứng. Đánh giá theo gói việc và thời gian thực tế, không đếm API hoặc commit.

Nguồn: đề dự án và rubric-midterm.pdf đã cung cấp; nội dung thống nhất với bộ chốt 1.0.0. Đây là kế hoạch công việc, chưa phải kết quả sản phẩm.

GÓI CÔNG VIỆC A

# Người A - Tài khoản, email và UI

Tự hoàn thành gói A với Payment mock; không cần đợi source, database hoặc báo cáo của B.

| Mã | Nhiệm vụ | Đầu ra / điều kiện hoàn thành |
|---|---|---|
| A1 | **Phân tích nghiệp vụ** | Use Case Diagram có file nguồn. Đặc tả đăng nhập, tra cứu, thanh toán, OTP, kết quả/lịch sử; điều kiện trước/sau và luồng lỗi. |
| A2 | **Thiết kế kiến trúc** | Architecture Diagram có frontend, 3 service, data store và email ngoài; bảng trách nhiệm, quyền sở hữu dữ liệu và cách giao tiếp. |
| A3 | **Account Service** | Database tài khoản, migration/seed, mật khẩu hash; API login và hồ sơ, kiểm tra token; request/response đúng OpenAPI. |
| A4 | **Notification Service** | API nội bộ, mẫu email OTP và thành công; kiểm tra X-Internal-Key, lưu messageId, xử lý gửi lặp/lỗi. Gửi được bằng request mẫu. |
| A5 | **Toàn bộ frontend** | Login, hồ sơ/số dư, tự tra cứu khi đủ MSSV, thanh toán toàn bộ, điều khoản, OTP, kết quả/lịch sử. Người nộp chỉ đọc; loading và lỗi rõ ràng. |
| A6 | **Mock và kiểm thử A** | Tự tạo Payment mock từ contract. Test Account/mail/UI; response tra cứu cũ không ghi đè MSSV mới. Lưu ảnh, email và kết quả kiểm thử. |
| A7 | **Đóng gói riêng A** | Source, Dockerfile/lệnh chạy, cấu hình mẫu, migration/seed, README A. Chạy được phần A cùng Payment mock trên máy mình. |
| A8 | **Báo cáo và slide A** | Viết đầy đủ phần A ở trang 4; có sơ đồ, ảnh, bảng API và kết quả riêng. Bàn giao nội dung hoàn chỉnh, không chỉ dàn ý. |

## API A chịu trách nhiệm

POST /api/v1/auth/login
GET /api/v1/users/me
POST /internal/v1/notifications/email
GET /api/v1/health trên Account và Notification

**Điều kiện bàn giao gói A**
Account và email chạy thật; UI chạy đủ luồng bằng mock; có test, source, cấu hình, README, báo cáo và slide. Khi ghép, A đổi UI sang Payment thật và bổ sung bằng chứng tích hợp.

GÓI CÔNG VIỆC B

# Người B - Thanh toán và dữ liệu

Tự hoàn thành Payment với token kiểm thử và Notification mock; không cần đợi UI hoặc đăng nhập của A.

| Mã | Nhiệm vụ | Đầu ra / điều kiện hoàn thành |
|---|---|---|
| B1 | **Thiết kế dữ liệu** | ERD có file nguồn, từ điển dữ liệu, khóa/quan hệ, constraints, trạng thái và quyền sở hữu dữ liệu. |
| B2 | **Database Payment** | Ví, sinh viên, học phí, giao dịch, OTP; migration/seed và persistence. Có ràng buộc số dư, requestId và thanh toán thành công duy nhất. |
| B3 | **API đọc dữ liệu** | Số dư, tra cứu MSSV, list/detail giao dịch đúng người dùng; schema và mã lỗi đúng OpenAPI. |
| B4 | **Tạo giao dịch và OTP** | Kiểm tra điều kiện, xử lý requestId; OTP gắn giao dịch, khác các mã đang hiệu lực, TTL 300 giây. Gọi Notification theo contract. |
| B5 | **Xác nhận thanh toán** | Khóa và kiểm tra lại OTP, số dư, công nợ. Cùng transaction: trừ tiền, gạch nợ, lịch sử, trạng thái, tiêu thụ OTP và tạo job email thành công. |
| B6 | **Transaction/concurrency** | Hai bài đồng thời; confirm/create lặp; lỗi giữa cập nhật phải rollback. Mail sau commit lỗi thì thử gửi lại, không chạy lại thanh toán. |
| B7 | **Mock và kiểm thử B** | Tự tạo JWT test đúng claim và Notification mock thu nhận OTP trong test. Test API, TTL/collision, tài chính, rollback và concurrency; lưu log/DB trước-sau. |
| B8 | **Đóng gói, báo cáo B** | Source, Dockerfile/lệnh chạy, config, README B, migration/seed, test; báo cáo đầy đủ và slide theo trang 4. |

## API B chịu trách nhiệm

GET /api/v1/wallet; GET /api/v1/tuition/{mssv}
POST và GET /api/v1/payments
GET /api/v1/payments/{paymentId}
POST /api/v1/payments/{paymentId}/confirm; GET /api/v1/health

**Điều kiện bàn giao gói B**
Backend và database chạy thật; đầy đủ OTP, rollback, idempotency và hai bài concurrency. Có test/log, source, cấu hình, README, báo cáo và slide; chạy riêng bằng mock/token test.

PHÂN CÔNG BÁO CÁO VÀ SLIDE

# Mỗi người viết phần mình từ đầu

A tổng hợp định dạng cuối. B kiểm tra lại dữ liệu và nội dung kỹ thuật. Cả hai bàn giao phần đã viết có hình, bảng, chú thích và bằng chứng.

| Chương báo cáo | Người A viết độc lập | Người B viết độc lập |
|---|---|---|
| **1. Giới thiệu, phạm vi** | Mục tiêu, đối tượng, chức năng/phi chức năng, giới hạn mô phỏng. | Giới hạn nghiệp vụ tài chính cần ghi nhận. |
| **2. Phân tích nghiệp vụ** | Use Case Diagram, đặc tả luồng chính/lỗi, trước/sau. | Quy tắc thanh toán, OTP và trạng thái giao dịch. |
| **3. Kiến trúc** | Architecture Diagram, trách nhiệm, giao tiếp và data ownership. | Ranh giới transaction và lý do gom dữ liệu tài chính ở Payment. |
| **4. Database** | Schema Account, migration/seed và constraints. | ERD, Payment schema, từ điển dữ liệu, constraints và persistence. |
| **5. REST API** | API Account và Notification: model, status, validation, errors. | API Payment: model, status, validation, errors. |
| **6. Hiện thực** | Account, email, UI, loading/validation/lỗi và ảnh màn hình. | Payment/OTP, sequence, cập nhật nguyên tử và email job. |
| **7. Transaction và concurrency** | Chạy kiểm chứng; bổ sung ảnh kết quả UI khi cần. | Cơ chế DB, thứ tự khóa, rollback, collision/idempotency và hai bài đồng thời. |
| **8. Kiểm thử** | Test Account/mail/UI; toàn luồng thật và email nhận được. | Test API/OTP/DB/concurrency; log và dữ liệu trước-sau. |
| **9. Cài đặt, sử dụng** | README A, cấu hình A, hướng dẫn UI; ghép README/Compose. | README B, lệnh DB/seed/reset/test; Dockerfile Payment; thử cài lại. |
| **10. Kết luận, đóng góp** | Kết quả/hạn chế A; tổng hợp định dạng, bảng đóng góp và slide. | Kết quả/hạn chế B; rà soát bằng chứng kỹ thuật và slide B. |

## Cách viết song song

**Trong lúc làm riêng:** viết thiết kế, code và kết quả test phần mình; ghi rõ bài nào dùng mock. **Khi ghép:** cập nhật ERD/kiến trúc theo code thật, bổ sung ảnh và kết quả tích hợp. Không ghi kết quả dự kiến thành kết quả đã đạt.

**Slide:** A trình bày yêu cầu, kiến trúc, tài khoản, UI và email. B trình bày dữ liệu, API Payment, OTP, transaction/concurrency và bằng chứng kiểm thử.

GHÉP BÀI VÀ NGHIỆM THU

# Chỉ ghép sau khi từng gói tự chạy

| STT | Người A làm | Người B làm |
|---|---|---|
| 1 | Bàn giao Account, Notification, frontend, test và báo cáo A. | Bàn giao Payment, database, test/log và báo cáo B. |
| 2 | Seed Account theo u1/u2; cấu hình email nhóm kiểm soát. | Seed ví/công nợ/lịch sử theo cùng ID; không trừ lại lịch sử mẫu. |
| 3 | Chạy Account 8081, Notification 8083, UI 5173; cấu hình xác thực chung. | Chạy Payment 8082; đồng bộ JWT_SECRET và INTERNAL_API_KEY. |
| 4 | Đổi Payment mock 8089 sang Payment thật 8082; cấu hình CORS phù hợp. | Đổi mail mock 8090 sang Notification thật 8083; dùng token Account thật. |
| 5 | Chạy UI toàn luồng, kiểm tra hai email và lưu ảnh thật. | Đối chiếu số dư/công nợ/lịch sử, chạy lại test và sửa lỗi nghiệp vụ. |
| 6 | Ghép định dạng báo cáo, ảnh UI, README/Compose và slide. | Ghép ERD/API/transaction/test; thử cài lại theo README tổng. |

## Hai bài đồng thời bắt buộc

**T21 - Cùng ví:** ví 10 triệu cùng trả HP001 và HP002, mỗi khoản 7 triệu. Chỉ một thành công; số dư cuối 3 triệu; chỉ một học phí được đóng.

**T22 - Cùng học phí:** u1 và u2 cùng trả HP001. Chỉ một giao dịch thành công, chỉ ví thắng bị trừ. **B viết/chạy test; A chạy lại và kiểm chứng.**

## Checklist bàn giao cuối

- Có source, migration/seed, test/log, README, cấu hình mẫu và Dockerfile/lệnh chạy.
- Bản demo dùng API/token/email thật; không còn mock trong luồng tích hợp.
- OTP, rollback, request lặp và hai bài concurrency có kết quả thực tế.
- Báo cáo, sơ đồ nguồn, ảnh, API docs và slide khớp cùng phiên bản code.
- Mỗi người giải thích được phần mình và hiểu toàn bộ luồng hệ thống.

**Tài liệu dùng chung:** 01-openapi.json; 02-du-lieu-mau.json; 03-ma-tran-kiem-thu.json; 04-cau-hinh.env.example trong bộ chốt 1.0.0. Ma trận có 37 case; chỉ đánh dấu đạt sau khi chạy thực tế.

Khi dùng Docker Compose, Payment gọi Notification bằng hostname notification; localhost trong container là chính container đó. Lỗi ở service nào do người phụ trách service đó sửa.